Problem 1

Latency vs. Throughput

Part A - Doubling the clock frequency

CPU execution time is T = (IC x CPI) / f, where IC is the dynamic instruction count, CPI is the average cycles per instruction, and f is the clock frequency.

Under the stated assumption that instruction count and execution behavior remain unchanged, IC and CPI are constant. Doubling frequency gives:

```
T_new = (IC x CPI) / (2f) = T_old / 2
```

Execution latency therefore decreases by 50%, giving a 2x speedup.

Instruction throughput is R = f / CPI. Therefore:

```
R_new = 2f / CPI = 2R_old
```

Instruction throughput doubles, while instructions per cycle remain unchanged.

Part B - Adding four identical CPU cores

A single-threaded database lookup executes on one core and cannot automatically distribute its instructions across the additional cores. Since each core's frequency and execution capabilities remain unchanged, the lookup's execution latency remains approximately unchanged, assuming unchanged resource contention.

A highly parallel web-server workload can distribute independent requests across the additional cores, increasing overall transaction throughput. If the original processor has N cores, the ideal throughput improvement is:

```
Throughput_new / Throughput_old = (N + 4) / N
```

For an original single-core processor, adding four cores gives five total cores and ideally 5x throughput. Actual improvement may be smaller because of shared memory bandwidth, synchronization, and I/O limitations.

Part C - Selecting the optimization target

The workload determines whether completing an individual task quickly or completing many tasks per second matters more.

A system with strict response deadlines should prioritize latency. For example, an automotive braking controller must process sensor inputs and issue a braking command within a short, predictable time.

A system handling many independent tasks should prioritize throughput. For example, a web server benefits from multiple cores and sufficient memory bandwidth to complete more client requests per second.

Problem 2

CMOS Dynamic Power Scaling and the Power Wall

Given values and power model

```
P_0 = 25 W; V_0 = 1.20 V; f_0 = 3.5 GHz
```

Dynamic power is P = alpha x C x V^2 x f. With constant activity factor alpha, the new-to-baseline power ratio is:

```
P_new / P_0 = (C_new / C_0) x (V_new / V_0)^2 x (f_new / f_0)
```

Part A - Power after shrinking to 7 nm

A 30% capacitance reduction gives C_new = (1 - 0.30)C_0 = 0.70C_0. Substituting V_new = 0.95 V and f_new = 4.0 GHz:

```
P_new = 25 x 0.70 x (0.95 / 1.20)^2 x (4.0 / 3.5)
= 25 x 0.70 x 0.626736 x 1.142857
P_new = 12.53 W (rounded)
```

The reduced capacitance and voltage more than offset the increase in frequency.

Part B - Power at the boosted frequency

The core remains on the same 7 nm node, so capacitance remains 0.70C_0. With V_boost = 1.15 V and f_boost = 4.8 GHz:

```
P_boost = 25 x 0.70 x (1.15 / 1.20)^2 x (4.8 / 3.5)
= 25 x 0.70 x 0.918403 x 1.371429
P_boost = 22.04 W (rounded)
```

Part C - Comparison and the Power Wall

```
Frequency increase = [(4.8 - 4.0) / 4.0] x 100 = 20%
Power increase = [(22.041667 / 12.534722) - 1] x 100 = 75.84%
```

A 20% frequency increase causes approximately a 75.84% increase in dynamic power, because the boost also requires higher voltage and power scales with V^2.

Supply voltage cannot decrease indefinitely while maintaining the required switching speed and reliable operation. Lowering transistor threshold voltage to support lower supply voltages also increases leakage. The assignment's approximate 1.0 V "voltage floor" describes this practical scaling limitation, rather than a universal minimum voltage.

Once voltage scaling becomes limited, increasing frequency raises power consumption and heat without compensating voltage reductions. Eventually, power delivery and cooling limits prevent further frequency increases. This is the physical origin of the Power Wall.

Problem 3

Speedup Limits and Amdahl's Law

Given values

```
Enhanced fraction f = 0.65; local speedup s = 40
Unenhanced fraction = 1 − f = 0.35
```

Part A - Overall system speedup

Applying Amdahl's law:

```
S = 1 / [(1 − f) + f / s]
= 1 / [(1 − 0.65) + 0.65 / 40]
= 1 / (0.35 + 0.01625)
= 1 / 0.36625
S = 2.73x (rounded to two decimal places)
```

For example, if baseline execution takes 100 seconds, the unenhanced portion still takes 35 seconds, while the matrix operations take 65 / 40 = 1.625 seconds. The new total execution time is therefore 36.625 seconds. This calculation assumes no additional NPU offloading overhead.

Part B - Maximum speedup with an infinitely fast NPU

As s approaches infinity, f / s approaches zero. Taking this limit in Amdahl's law gives:

```
S_max = 1 / (1 − f)
= 1 / 0.35
S_max = 2.86x (rounded to two decimal places)
```

Even an infinitely fast NPU cannot eliminate the remaining 35% of baseline execution time.

Part C - Bottleneck and making the common case fast

The bottleneck is the 35% of execution time that the NPU does not accelerate. This may include CPU control logic, data preparation, or other non-matrix operations. As matrix execution becomes faster, this unchanged work dominates total runtime and prevents speedup from exceeding approximately 2.86x.

Architects prioritize making the common case fast because the overall benefit depends on the fraction of execution time affected. A large speedup for a rarely executed operation saves little total time, whereas improving a major contributor to runtime can substantially improve overall performance.

In this workload, further substantial gains require reducing the remaining 35% of execution time.